

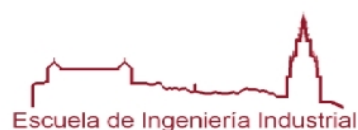
Práctica 2: Programación del puerto RS-232 en Python.

Comunicación de dos PCs

Escuela de Ingeniería Industrial de Toledo

Autor

Juan Moreno García



Índice general

2. Programación RS-232 en C	2
2.1. Objetivos.	2
2.2. Librería Serial.	2
2.2.1. Función <i>Serial</i>	3
2.2.2. Función <i>write(cadena)</i>	4
2.2.3. Función <i>readline()</i>	4
2.2.4. Función <i>close()</i>	5
2.3. Ejemplos.	5
2.3.1. Comunicación mediante texto entre dos aplicaciones Python.	5
2.3.2. Envío y recepción de peticiones de acciones.	7

2

Programación RS-232 en C

2.1. Objetivos.

El lenguaje Python dispone de la librería *serial* para la programación del puerto serie RS-232. Para utilizar esta librería hay que incluirla en los programas mediante la orden “*import serial*”. Al final de la práctica el alumno debe ser capaz de:

1. Programar en Python la comunicación RS-232 utilizando la librería de “*serial*”.
2. Enviar información entre aplicaciones Python vía RS-232, así como realizar los cambios de formatos necesarios entre los diferentes tipos (int, float, etc) a partir de la información enviada y recibida.

2.2. Librería Serial.

Esta librería permite realizar transferencias entre dos dispositivos conectados vía RS232. En la dirección web “<http://pythonhosted.org/pyserial/>” se encuentra la documentación de *serial* en la que se detallan todas las funciones de Python para su uso. Las cuatro funciones a utilizar son las siguientes:

1. *Serial()*: abre un puerto serie para realizar el intercambio de información a través de él. Sólo lo puede tener una aplicación simultáneamente conectada al mismo puerto.
2. *write()*: Transferir una cadena de bytes por un puerto. Usualmente su último carácter será ‘\n’, ya que facilita la lectura de la secuencia recibida tanto desde otro programa Python, como desde un *Arduino* o *nodeMCU*.
3. *readline()*: Leer una cadena terminada en ‘\n’.

4. `close()`: Cierra un puerto serie. Es muy importante terminar los programas implementados cerrando los recursos correctamente, ya que en el caso de los puertos es necesario liberar el puerto para que otra aplicación lo pueda utilizar de nuevo.

A continuación se detallará el funcionamiento de las funciones.

2.2.1. Función *Serial*.

La función para abrir un puerto es *Serial* y tiene varios parámetros en su uso. En la dirección http://pythonhosted.org/pyserial/pyserial_api.html?highlight=serial.serialserial.Serial se detalla todos los parámetros y su significado. Tiene la siguiente estructura:

```
# Abre el puerto serie
PuertoSerie = serial.Serial(
    port = 'COM1', # "COM1", "COM2", ....
    baudrate = 9600, # 9600, 19200, ...
    parity = serial.PARITY_NONE, # PARITY_NONE, PARITY_EVEN, PARITY_ODD
    stopbits = serial.STOPBITS_ONE, # STOPBITS_ONE, STOPBITS_TWO
    bytesize = serial.EIGHTBITS # FIVEBITS, SIXBITS, SEVENBITS, EIGHTBITS
)
```

Los parámetros son:

Port: Indica que puerto es el que se desea abrir. Es una cadena del tipo “COM1”, “COM2”, ...

baudrate: Tasa de transferencia de baudios. Las etiquetas a usar serán: 9600, 19200, 56000, etc.

bytesize: Número de bits transmitidos. Las posibilidades son entre 5 y 8. Las etiquetas a usar son *FIVEBITS*, *SIXBITS*, *SEVENBITS* o *EIGHTBITS*. En las prácticas se utilizará *EIGHTBITS*, es decir, un byte.

parity: Paridad utilizada para la detección de errores. Las posibles etiquetas son *PARITY_NONE*, *PARITY_EVEN* y *PARITY_ODD*. En las prácticas no se utilizará comprobación de paridad, etiqueta *PARITY_NONE*.

stopbits: Número de bits de stop. Los valores más usuales son *STOPBITS_ONE* y *STOPBITS_TWO*. En las prácticas se utilizará un bit de stop (*STOPBITS_ONE*).

timeout: Permite definir un timeout que indica el tiempo máximo que se esperará. En nuestro caso no se utilizará *timeout*. El valor *None* espera hasta que llegue algo.

Esta función devuelve un objeto que permite manejar el puerto. A este objeto lo vamos a llamar “manejador”, y es el identificador del puerto que se usa dentro del programa. En el resto de funciones, las operaciones sobre el puerto se realizan utilizando dicho manejador.

Un ejemplo de invocación de esta función sobre el puerto “COM8”, a una tasa de 19200, con paridad par, utilizando dos bits de stop y utiliza un tamaño de datos de 6 bits es:

```
# Abre el puerto COM8
PuertoSerie = serial.Serial(
    port = 'COM8',
    baudrate = 19200,
    parity = serial.PARITY_ODD,
    stopbits = serial.STOPBITS_TWO,
    bytesize = serial.SIXBITS,
    timeout = None
)
```

2.2.2. Función *write(cadena)*.

La función para enviar una cadena de bytes por un puerto es *write* y tiene la siguiente estructura:

```
# PuertoSerie es el manejador del puerto (objeto)
PuertoSerie.write( cadena )
```

Su parámetro es la cadena de bytes a enviar, y la función devuelve el número de bytes enviados por el puerto. El manejador que lo lleva se pone delante junto con un punto.

Este ejemplo muestra cómo enviar una cadena de bytes (“*esto es un mensaje*”) terminada en ‘\n’:

```
cad = 'esto es un mensaje' # cadena de caracteres
# cad1 se codifica como cadena de bytes que es el formato a enviar
# encode( ) realiza la codificación como cadena de bytes
# se añade '\n' para que en destino se pueda usar readline( )
cad1 = (cad+'\\n').encode('utf-8')
PuertoSerie.write( cad1 ) # hace el envío
```

2.2.3. Función *readline()*.

Es una función devuelve la cadena de bytes finalizada en ‘\n’ recibida por el puerto y tiene la siguiente estructura:

```
# r contendrá la cadena en formato cadena de bytes leída
r = PuertoSerie.readline()
```

El manejador del puerto se coloca delante de la función que recibe la cadena de bytes. Para trabajar correctamente en los programas la cadena recibida se debe decodificar utilizando la función *decode()*.

El siguiente ejemplo muestra cómo recibir una cadena de bytes por un puerto (el asociado a *PuertoSerie*) y convertirla a una lista de *n float*. La cadena recibida tendrá la forma $n : v_1, \dots, v_n$, siendo n el número de valores *float* que vendrán a continuación separados por comas. A partir de la recepción de la cadena ésta se puede procesar con las funciones habituales de cadenas.

```
# r contendrá la cadena en formato cadena de bytes leída
r = PuertoSerie.readline()
# r1 la pasa a string de python, se quita el último carácter
r1 = r.decode()[0:-1]
# r2 es una lista obtenida como la cadena partida utilizando ':'
r2 = r1.split(':')
# se almacena n
n = int(r2[0])
# r3 es una lista obtenida a partir de r2[1] como la cadena
# partida utilizando ','
r3 = r2[1].split(',')
# s es una lista con dos componentes, el primero un entero que indica
# el número de floats contenidos en su segunda componente
s = [ float(e) for e in r3 ]
```

Se recibe una cadena de bytes que se almacena en *r*, por ejemplo *b'2:2.3,2.4'*. A continuación *r* se decodifica utilizando la función *decode()* y desechando el último carácter, almacenando el resultado en *r1*. Ahora se utiliza la función *split()* dos veces, la primera sobre *r1* que trocea *r1* obteniendo *r2*, y la segunda sobre el segundo componente de *r2* que tiene sus valores separados por ',', el resultado se almacena en *r3*. La variable *n* almacena el número de elementos *float* de la cadena. Finalmente ya se puede componer la lista final de *float* que se calcula con la orden *[float(e) for e in r3]*.

2.2.4. Función *close()*.

La función para cerrar un puerto es *close()* y tiene la siguiente estructura:

```
PuertoSerie.close()
```

La sentencia *close()* debe ejecutarse obligatoriamente en todo programa para liberar el puerto, ya que sólo una aplicación puede disponer de él simultáneamente. El programa debe estar correctamente implementado para que en todos los casos se ejecute la orden para cerrar el puerto.

2.3. Ejemplos.

A continuación se mostrarán dos ejemplos que utilizan el puerto RS-232 para comunicar dos programas *Python*. En caso de no poder realizar la conexión física en un único PC o la conexión de dos es necesario el uso un programa que permita simular puertos virtuales, como por ejemplo *VSPE*, presentado en la práctica 1.

2.3.1. Comunicación mediante texto entre dos aplicaciones *Python*.

A continuación se presentarán dos programas (programas uno y dos)que intercambian texto alternativamente. Inicialmente, el Programa uno solicita por teclado una cadena de texto, le añade '\n' como último carácter, y la envía al programa dos, quedando a la espera

de la recepción de un mensaje proveniente del programa dos terminado en '\n'. El programa dos está a la espera de recibir una cadena que finaliza con '\n' proveniente del programa uno, la muestra por pantalla y espera la introducción por teclado de otra cadena de texto que será enviada al programa uno.

Programa uno

```
# Importamos la libreria de PySerial
import serial

# Abrimos el puerto del arduino a 9600
PuertoSerie = serial.Serial(
    port='COM1', # cambiar al puerto correcto
    baudrate=9600,
    parity=serial.PARITY_NONE,
    stopbits=serial.STOPBITS_ONE,
    bytesize=serial.EIGHTBITS
)

# Creamos un bucle sin fin
s = '' # para la entrada
r = '' # para la salida
while s!='fin' and r!='fin':
    s = input('Envia un comentario: ')
    PuertoSerie.write( (s+'\n').encode('utf-8') )
    if s!='fin':
        print( 'Esperando recibir ... ' )
        r = PuertoSerie.readline()
        r = r.decode()[0:-1]
        print( 'Respuesta recibida: ', r )
PuertoSerie.close()
```

Programa dos

```
# Importamos la libreria de PySerial
import serial

# Abrimos el puerto del arduino a 9600
PuertoSerie = serial.Serial(
    port='COM2', # Cambiar al puerto correcto
    baudrate=9600,
    parity=serial.PARITY_NONE,
    stopbits=serial.STOPBITS_ONE,
    bytesize=serial.EIGHTBITS
)

s = ''
r = ''
while s!='fin' and r!='fin':
    print( 'Esperando recibir ...' )
    r = PuertoSerie.readline()
```

```

r = r.decode()[0:-1]
print( 'Cadena recibida:', r )
if r!='fin':
    s = input('Envia un comentario: ')
    PuertoSerie.write( (s+'\n').encode('utf-8') )
PuertoSerie.close()

```

2.3.2. Envío y recepción de peticiones de acciones.

Este programa tiene una arquitectura cliente-servidor. El programa cliente envía peticiones al servidor, y éste realiza la petición y le devuelve la respuesta si procede. En este ejemplo se mostrará el envío y recepción de peticiones para leer/escribir los componentes de una lista de tres componentes (dos enteros y un real). La lista tiene la estructura:

```

# la primera componentes es la variable a
# la segunda componente es b
# la tercera componente es result
struct = [0, 0, 0.0]

```

Las componentes *a* y *b* son dos enteros que podrán ser asignados mediante sendas peticiones. La componente *result* tomará el valor según la petición recibida: máximo de *a* o *b*, mínimo de *a* o *b*, o media de *a* o *b*. Las posibles actuaciones sobre la estructura son:

1. Asignar atributo *a*.
2. Asignar atributo *b*.
3. Asignar atributo *result* con el mínimo de *a* o *b*.
4. Asignar atributo *result* con el máximo de *a* o *b*.
5. Asignar atributo *result* con la media de *a* o *b*.
6. Devolver al cliente el valor *result*.
7. Devolver al cliente la lista completa.

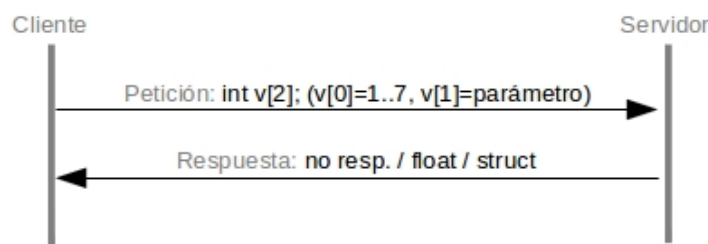


Figura 2.1: Mensajería del ejemplo.

La Figura 2.1 muestra cómo funciona la mensajería entre cliente y servidor. Para la comunicación de las peticiones del cliente al servidor se utiliza una cadena llamada *v* con la estructura $v_0:v_1$, es decir, está formada por dos componentes separados por ':', donde cada componente toma los siguientes valores:

- v_0 : Un valor entero en el intervalo cerrado 1 y 7 que representa la petición.
- v_1 : Valor del parámetro de la petición si es necesario. Este parámetro sólo se utiliza en las peticiones 1 y 2 para indicar el valor a asignar en a o b respectivamente.

El servidor sólo debe responder a 2 de las 7 peticiones:

1. En la petición 6 devuelve un *float* al cliente que es el valor del atributo *result* de la estructura.
2. En la petición 7 devuelve la estructura completa.

Para resolver este problema se han programado dos librerías. La primera de ella contiene funciones para manejar la estructura *struct* (*struc.py*). La otra está compuesta por tres funciones para manejar el puerto (*mensaje.py*).

A continuación se muestran las funciones contenidas en *struc.py*.

Archivo *struc.py*

```
def setA(struct, a):
    struct[0] = a

def setB(struct, b):
    struct[1] = b

def setResult(struct, r):
    struct[2] = r

def minimo(struct):
    setResult(struct, float( min(struct[0:2]) ) )

def maximo(struct):
    setResult(struct, float( max(struct[0:2]) ) )

def media(struct):
    setResult(struct, sum(struct[0:2])/2.0 )

def getResult(struct):
    return struct[2]
```

Como puede verse, son funciones que leen o escriben la componente relativa a las variables a , b o *result*.

Archivo *mensaje.py*

```
import serial

def abrirPort(p):
    return serial.Serial(
        port = p,
```

```

        baudrate = 115200,
        parity = serial.PARITY_NONE,
        stopbits = serial.STOPBITS_ONE,
        bytesize = serial.EIGHTBITS
    )

def sendMensaje(ps,s):
    ps.write( s.encode('utf-8') )

def receiveMensaje(ps):
    r = ps.readline()
    return r.decode()[0:-1]

```

Como puede verse, está compuesto de tres funciones para abrir un puerto, enviar un mensaje y recibir un mensaje. Las funciones están compuestas por las órdenes descritas en la Sección 2.2.

A continuación se presenta los programas cliente y servidor.

Programa Cliente (*cliente.py*)

```

from mensaje import *

def menu():
    print("0) Salir")
    print("1) Asignar A")
    print("2) Asignar B")
    print("3) Minimo")
    print("4) Maximo")
    print("5) Media")
    print("6) Recibir float resultado")
    print("7) Recibir struct completo")
    return input('Elige una opción: ')

# Abrimos el puerto del arduino a 9600
PuertoSerie = abrirPort('COM1') # cambiar al correcto

# Creamos un bucle que finaliza con '0'
opc = ''
while opc!='0':
    opc = menu()
    mensa = ''
    if opc=='0':
        mensa = '0\n'
        sendMensaje(PuertoSerie, mensa)
    elif opc == '1' or opc == '2':
        v = input('Dame el nuevo valor: ')
        mensa = opc + ':' + v + '\n'
        sendMensaje(PuertoSerie, mensa)
    elif opc >= '3' and opc <= '5':
        mensa = opc + '\n'

```

```

        sendMensaje(PuertoSerie, mensa)
    elif opc=='6' or opc=='7':
        mensa = opc + '\n'
        sendMensaje(PuertoSerie, mensa)
        # leer float o estructura desde servidor
        r = receiveMensaje(PuertoSerie)
        print( 'struct =', r )

```

Programa Servidor (servidor.py)

```

import struct as st
from mensaje import *

# Abrimos el puerto del arduino a 115200
PuertoSerie = abrirPort('COM2') # cambiar al puerto apropiado
# [a, b, result]
struct = [0, 0, 0.0]
r = [-1] # a -1 para que no sea 0 y entre en el while
while r[0]!='0':
    print( 'Esperando petición ...' )
    r = PuertoSerie.readline()
    r = r.decode()[0:-1]
    r = r.split(':')
    print( r )
    if r[0]=='1': # Asignar A
        st.setA(struct, int(r[1]))
    elif r[0]=='2': # Asignar B
        st.setB(struct, int(r[1]))
    elif r[0]=='3': # minimo
        st.minimo(struct)
    elif r[0]=='4': # maximo
        st.maximo(struct)
    elif r[0]=='5': # media
        st.media(struct)
    elif r[0]=='6': # peticion de result
        sendMensaje(PuertoSerie, str(st.getResult(struct))+'\n' )
    elif r[0]=='7': # envio de la estructura completa
        sendMensaje(PuertoSerie, str(struct)+'\n' )
    print('struct =', struct)

```